

Rutas Mínimas en Grafos

Dijkstra & Bellman-Ford

Técnica de relajación de aristas aplicada a tres grafos dirigidos y ponderados para determinar el camino de menor costo desde el nodo inicio hasta el nodo fin.

- Enrique Jara Escobar
- Sergio Alejandro Ariza
- Andres Ricardo Poveda

1. Descripción del Problema

Se tienen tres grafos dirigidos y ponderados donde cada nodo representa un punto de la red y cada arista tiene un peso (costo de transitar ese tramo). El objetivo es determinar, para cada grafo, cuál es el peso total de la ruta más corta que va del nodo inicio al nodo fin.

Grafo	Nodos	Aristas	Pesos negativos	Algoritmo
1	6 (0-5)	9	No	Dijkstra
2	5 (0-4)	5	No	Dijkstra
3	5 (0-4)	6	Sí (-1)	Bellman-Ford

Técnica de Relajación

Ambos algoritmos comparten el mismo núcleo: **relajar aristas**. Dado un nodo u con distancia estimada $dist[u]$ y una arista $(u, v, peso)$:

```
si dist[u] + peso < dist[v]:
    dist[v] = dist[u] + peso # actualizar distancia
    padre[v] = u # registrar nodo padre
```

La diferencia está en **cuándo y cuántas veces** se aplica la relajación: Dijkstra la aplica en orden de menor distancia (heap), Bellman-Ford la aplica $|V|-1$ veces sobre todas las aristas.

2. Requerimientos

Requerimientos Funcionales

RF01	Representación del grafo como diccionario de adyacencia sin usar clases.
RF02	Relajación Dijkstra con cola de mínimos (heapq); cada nodo se procesa una sola vez.
RF03	Relajación Bellman-Ford: $ V -1$ rondas sobre todas las aristas; admite pesos negativos.
RF04	Reconstrucción del camino óptimo siguiendo punteros padre desde fin hasta inicio.
RF05	Impresión del costo mínimo y la secuencia de nodos del camino para cada grafo.

Requerimientos No Funcionales

RNF01	Dijkstra en $O((V+E) \log V)$ usando min-heap.
RNF02	Bellman-Ford en $O(V \cdot E)$ iterando $ V -1$ rondas.
RNF03	Solo Python puro y heapq de la biblioteca estándar; sin librerías de grafos.
RNF04	Sin clases; implementación con funciones y estructuras nativas (dict, list).
RNF05	Compatible con Python 3 sin instalaciones adicionales.
RNF06	Bellman-Ford detecta y alerta ciclos de peso negativo.

3. Historias de Usuario

HU01	Como estudiante, quiero representar cada grafo como un diccionario de Python. <i>Criterio:</i> El diccionario contiene exactamente los nodos y aristas del diagrama con sus pesos correctos. (RF01)
HU02	Como analista, quiero ejecutar Dijkstra sobre grafos sin pesos negativos. <i>Criterio:</i> El algoritmo termina en $O((V+E)\log V)$ y devuelve el costo y camino correctos. (RF02)
HU03	Como analista, quiero ejecutar Bellman-Ford sobre el Grafo 3 (arista peso -1). <i>Criterio:</i> El algoritmo itera $ V -1$ veces y devuelve la distancia mínima correcta. (RF03)
HU04	Como usuario, quiero ver la secuencia de nodos del camino óptimo. <i>Criterio:</i> La reconstrucción sigue los punteros padre desde fin hasta inicio e invierte la lista. (RF04)
HU05	Como supervisor, quiero ver el peso total de cada ruta mínima. <i>Criterio:</i> El sistema imprime Costo mínimo: X con el valor numérico correcto. (RF05)

4. Análisis de Complejidad

Dijkstra — Grafos 1 y 2

Operación	Mejor	Promedio	Peor	Razón
Inicializar dist	$O(V)$	$O(V)$	$O(V)$	Un valor por vértice
Extraer mínimo (heap)	$O(\log V)$	$O(\log V)$	$O(\log V)$	Operación heap
Relajar todas las aristas	$O(E \log V)$	$O(E \log V)$	$O(E \log V)$	E relajaciones $\times$ heap
Total Dijkstra	$O((V+E)\log V)$	$O((V+E)\log V)$	$O((V+E)\log V)$	

Bellman-Ford — Grafo 3

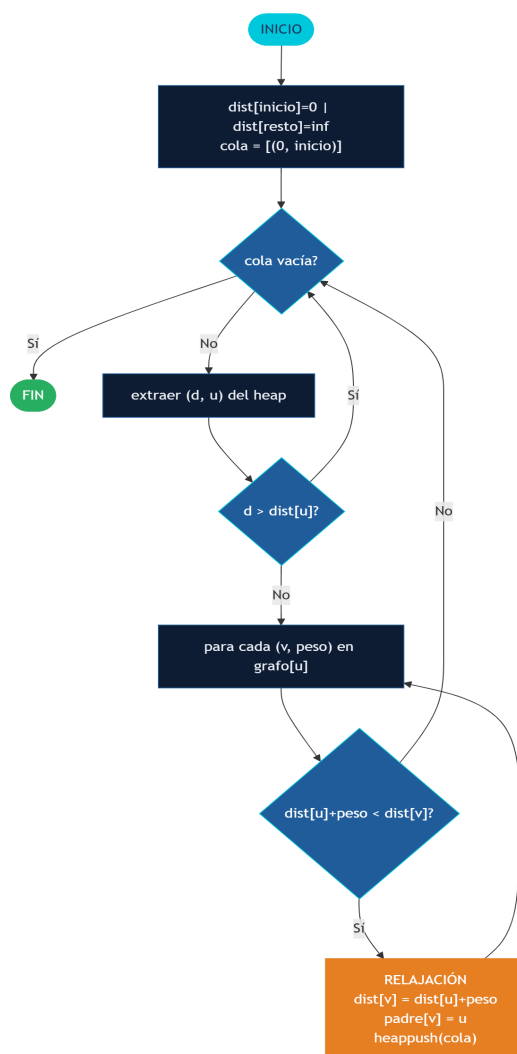
Operación	Mejor	Promedio	Peor	Razón
Inicializar dist	$O(V)$	$O(V)$	$O(V)$	Un valor por vértice
Ronda de relajación	$O(E)$	$O(E)$	$O(E)$	Recorre E aristas
Número de rondas	$O(1)^*$	$O(V)$	$O(V)$	*Sin cambios: termina antes
Verificar ciclo negativo	$O(E)$	$O(E)$	$O(E)$	Una pasada extra
Total Bellman-Ford	$O(V \cdot E)$	$O(V \cdot E)$	$O(V \cdot E)$	

Complejidad Espacial

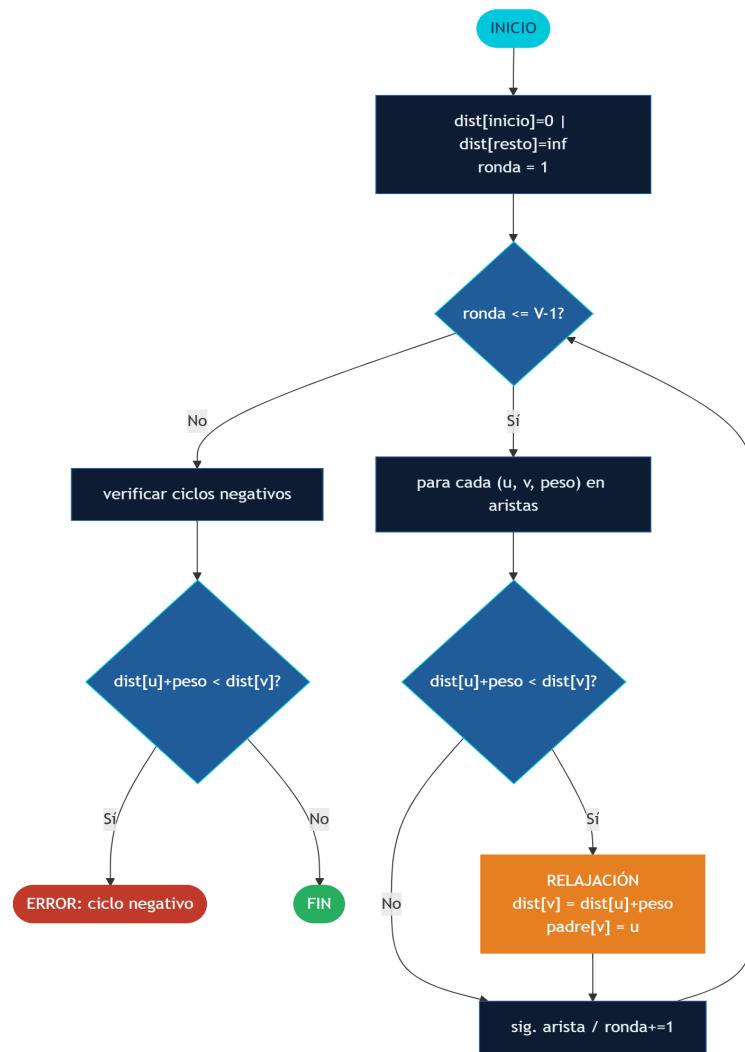
Estructura	Espacio
Diccionario de adyacencia	$O(V + E)$
Arreglo de distancias dist	$O(V)$
Diccionario de padres	$O(V)$
Cola de prioridad (Dijkstra)	$O(E)$ peor caso
Total espacial	$O(V + E)$

5. Diagramas de Flujo

Dijkstra



Bellman-Ford



6. Funciones Base de Relajación

Implementación sin clases, usando únicamente Python puro y heapq.

Dijkstra

```
import heapq

def dijkstra(grafo, inicio):
    dist = {n: float('inf') for n in grafo}
    padre = {n: None for n in grafo}
    dist[inicio] = 0
    cola = [(0, inicio)]
    while cola:
        d, u = heapq.heappop(cola)
        if d > dist[u]: continue
        for v, peso in grafo[u]:
            nueva = dist[u] + peso
            if nueva < dist[v]: # RELAJACIÓN
                dist[v] = nueva
                padre[v] = u
            heapq.heappush(cola, (nueva, v))
    return dist, padre
```

Bellman-Ford

```
def bellman_ford(aristas, nodos, inicio):
    dist = {n: float('inf') for n in nodos}
    padre = {n: None for n in nodos}
    dist[inicio] = 0
    for _ in range(len(nodos) - 1):
        for u, v, peso in aristas:
            if dist[u] + peso < dist[v]: # RELAJACIÓN
                dist[v] = dist[u] + peso
                padre[v] = u
    for u, v, peso in aristas:
        if dist[u] + peso < dist[v]:
            raise ValueError('Ciclo de peso negativo detectado')
    return dist, padre
```

Reconstrucción del Camino

```
def reconstruir_camino(padre, inicio, fin):
    camino, nodo = [], fin
    while nodo is not None:
        camino.append(nodo)
        nodo = padre[nodo]
    camino.reverse()
    return camino if camino and camino[0] == inicio else []
```


7. Ejercicio 1 — Grafo de 6 Nodos (Dijkstra)

inicio = 0 · fin = 5

Arista	Peso	Arista	Peso
0 → 1	5	2 → 1	8
0 → 2	2	2 → 4	7
1 → 3	4	3 → 5	3
1 → 4	2	3 → 4	6
		4 → 5	1

Traza de Relajación — Dijkstra

Paso	Nodo proc.	dist[1]	dist[2]	dist[3]	dist[4]	dist[5]
0	—	∞	∞	∞	∞	∞
1	0	5	2	∞	∞	∞
2	2	5	—	∞	9	∞
3	1	—	—	9	7	∞
4	4	—	—	9	—	8
5	3	—	—	—	—	8

Camino óptimo: 0 → 1 → 4 → 5	Costo = 8
------------------------------	-----------

8. Ejercicio 2 — Grafo de 5 Nodos (Dijkstra)

inicio = 0 · fin = 4

Arista	Peso
0 → 1	10
1 → 2	20
1 → 3	1
3 → 2	1
2 → 4	30

Traza de Relajación — Dijkstra

Paso	Nodo proc.	dist[1]	dist[2]	dist[3]	dist[4]
0	—	∞	∞	∞	∞
1	0	10	∞	∞	∞
2	1	—	30	11	∞
3	3	—	12	—	∞
4	2	—	—	—	42

Camino óptimo: 0 → 1 → 3 → 2 → 4	Costo = 42
----------------------------------	------------

9. Ejercicio 3 — Grafo con Peso Negativo (Bellman-Ford)

inicio = 0 · fin = 4 — Arista 3→1 con peso -1 requiere Bellman-Ford.

Arista	Peso
0 → 2	2
0 → 1	2
2 → 4	2
2 → 3	2
3 → 1	-1 ■
3 → 4	2

Traza Ronda 1 — Bellman-Ford

Arista relajada	dist[0]	dist[1]	dist[2]	dist[3]	dist[4]
Estado inicial	0	∞	∞	∞	∞
0→2 (w=2)	0	∞	2	∞	∞
0→1 (w=2)	0	2	2	∞	∞
2→4 (w=2)	0	2	2	∞	4
2→3 (w=2)	0	2	2	4	4
3→1 (w=-1) ■	0	3	2	4	4
3→4 (w=2)	0	3	2	4	4
Rondas 2-4	0	3	2	4	4 (sin cambios)

Camino óptimo: 0 → 2 → 4	Costo = 4
--------------------------	-----------

10. Tests

Suite de 18 pruebas automatizadas ejecutadas sobre los tres grafos.

ID	Descripción	Resultado
T01	Grafo 1: costo mínimo = 8	PASS
T02	Grafo 1: camino = [0,1,4,5]	PASS
T03	Grafo 1: dist[2] = 2	PASS
T04	Grafo 1: dist[4] = 7	PASS
T05	Grafo 2: costo mínimo = 42	PASS
T06	Grafo 2: camino = [0,1,3,2,4]	PASS
T07	Grafo 2: dist[3] = 11	PASS
T08	Grafo 2: dist[2] = 12	PASS
T09	Grafo 3: costo mínimo = 4	PASS
T10	Grafo 3: camino = [0,2,4]	PASS
T11	Grafo 3: dist[2] = 2	PASS
T12	Grafo 3: dist[3] = 4	PASS
T13	Grafo 3: peso -1 relaja dist[1] = 3	PASS
T14	Dijkstra: nodo único dist[0] = 0	PASS
T15	Bellman-Ford: sin aristas dist[1]=inf	PASS
T16	reconstruir sin ruta = []	PASS
T17	Dijkstra: inicio=fin → dist = 0	PASS
T18	BF detecta ciclo negativo	PASS

Total:	18 PASS	0 FAIL
--------	---------	--------